

Confidential

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

표적 궤적 모의 TargetSim

Janghwan Kim (Harley)

01

표적을 정의한다

표적 · 기동을 어떤 값으로
적을 것인가

5 ~ 8

02

시간에 따라 움직인다

한 스텝에 쓰는 공식과
그 공식이 필요한 이유

9 ~ 13

03

결과를 확인한다

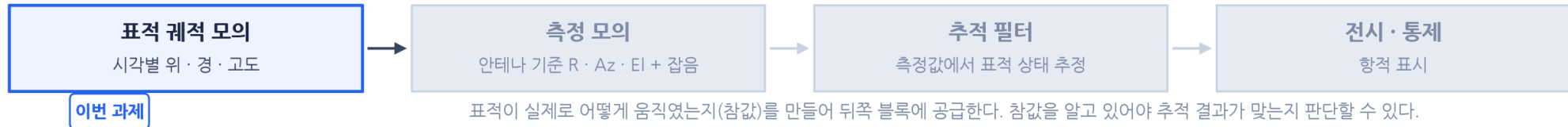
60 초 궤적 · 기동 시연
운용 화면

14 ~ 17

무엇을 왜 만들었는가

표적이 실제로 어떻게 움직였는지 — 참값 — 를 만들어 내는 일입니다. 참값이 있어야 추적 결과가 맞는지 판정할 수 있습니다.

이 모의가 어디에 쓰이는가



구현은 세 단계로 나뉘었다



만든 것 — 계산과 화면을 나눈다

연산 엔진만 따로 빌드해 검증할 수 있게 했습니다. 오늘 보여 드리는 숫자는 모두 그 엔진을 돌려 얻은 것입니다.



01. 표적을 정의한다

표적을 어떤 값으로 적을 것인가

명세가 정한 7 개를 받아, 계산하기 좋은 9 개로 바꿔 들고 있습니다.

받는 값 — 명세가 정한 7 개

위도	32.12 °
경도	126.00 °
고도	300 m
속력	200 m/s
Roll	0 °
Pitch	0 °
Yaw	180 °

어디에 · 어디를 보고 · 얼마나 빠르냐.
속도는 크기 하나만 받는다 — 방향은
자세각이 이미 들고 있기 때문이다.

초기화

내부에서 굴리는 값 — 상태 9 개

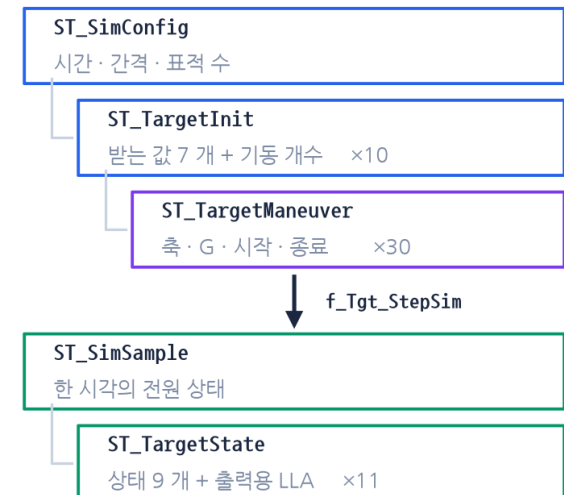
- $\mathbf{p}^e$ ECEF 위치
 x, y, z [m]
- $\mathbf{v}^e$ ECEF 속도
 v_x, v_y, v_z [m/s]
- Θ 자세각
Roll, Pitch, Yaw [rad]

왜 ECEF 로 바꿔 두는가

- 미터 단위 직교좌표라 위치 전진이
덧셈 한 번으로 끝난다
- 명세가 지정한 갭신 좌표계다

출력할 때만 LLA 로 되돌린다.
좌표변환 함수는 Part 2 의 것을 그대로 쓴다.

구조체로 옮기면



입력과 출력을 완전히 같았다. 설정은 처음
한 번만 읽고, 스텝마다 바뀌는 것은 상태뿐이다.

10 × 30 은 고정 배열 — 실행 중 메모리 할당 실패가 없다.

01. 표적을 정의한다

기동을 어떤 값으로 적을 것인가

"12 초부터 16 초까지 오른쪽으로 8 G" 라는 지시가 구조체 한 줄이 됩니다.

기동 1 건 = 값 네 개

enTurnType Roll / Pitch / Yaw	어느 축으로 도는가
gravityValue 크기 = 세기, 부호 = 방향 [G]	얼마나 세게 도는가
startTime 이 시각 포함 [s]	언제부터
endTime 이 시각 직전까지 [s]	언제까지

기동이 없는 시각에는 각속도 0 — 직진한다.
구간이 서로 겹치면 계산 전에 걸러낸다.

기동 지시는 원래 "위도 32.1 에서 좌로 90 도" 처럼 좌표로 온다.
구조체에는 시각 칸만 있으므로, Core 를 한 번 돌려 그 좌표에
닿는 시각을 찾아 start / end 로 옮겨 적었다.

왜 G 하나로 충분한가

필요한 것은 "1 초에 몇 도 도는가" 인데, 받는 값은 G 하나다.

$$a_c = \frac{V^2}{R} = V\omega = n g$$

원을 그리며 돌 때 필요한 가속도를 중력의 n 배로 적는다

$$\omega = \frac{n g}{V} \quad \rightarrow \text{이 한 줄로 각속도가 정해진다}$$

`omega = gravityValue * G_FORCE / headingSpeed;`

같은 4 G 라도

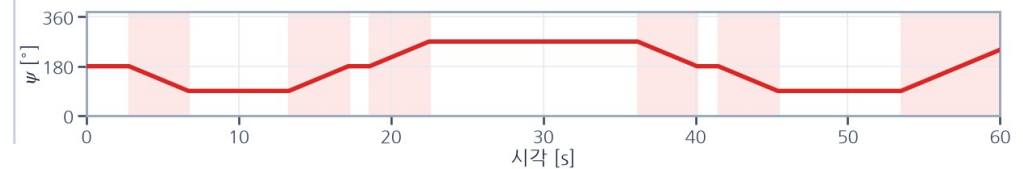
대함 30 m/s 74.9 °/s R 23 m

대공 200 m/s 11.2 °/s R 1020 m

분모에 속력이 있어 빠른 표적일수록 덜 꺾인다.

기동표 6 줄이 실제로 그려낸 방위각

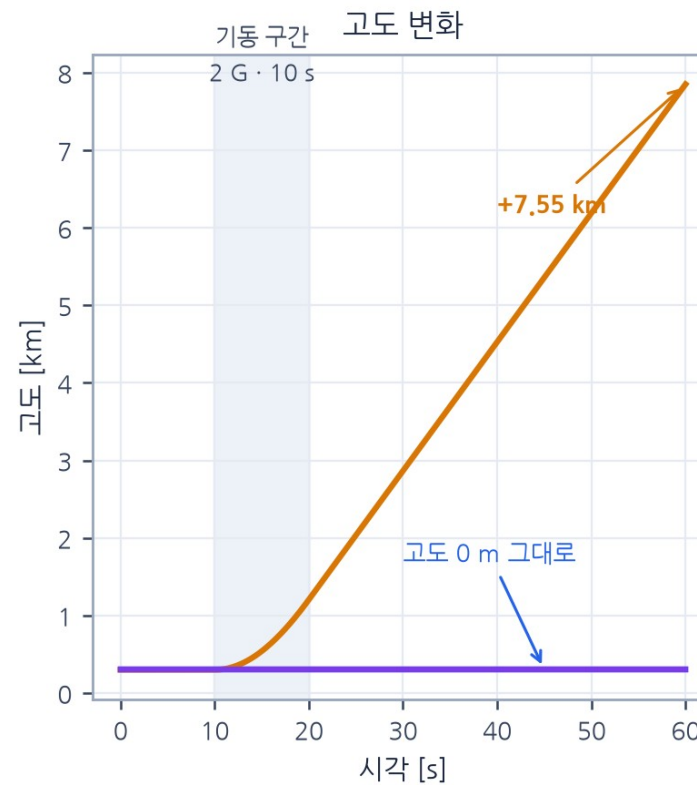
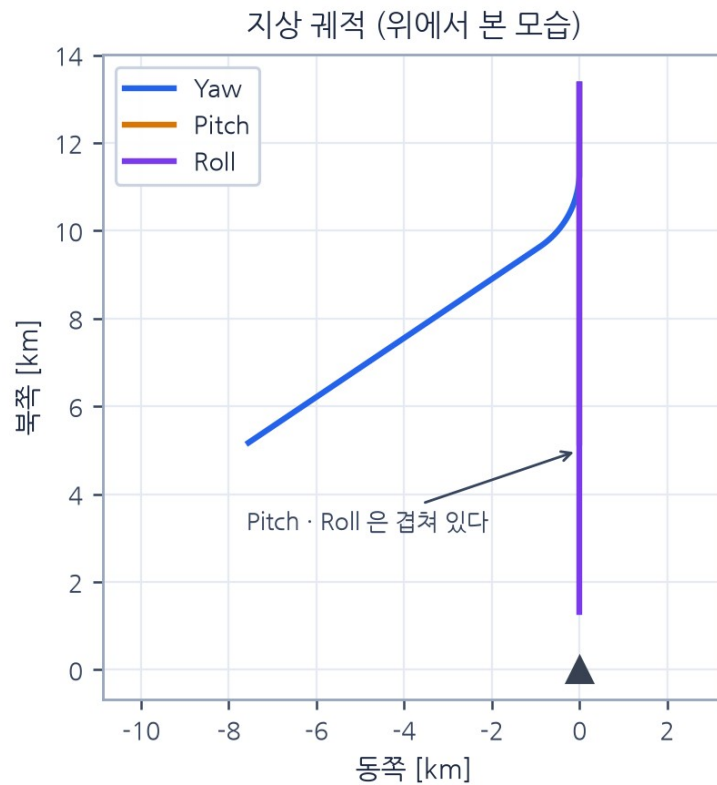
색칠한 구간에서만 각이 돈다



01. 표적을 정의한다

축을 바꾸면 궤적이 어떻게 달라지는가

같은 표적에 같은 2 G 를 10 초씩, 축만 바꿔 걸어 본 실험 결과입니다.



같은 표적 · 같은 2 G · 같은 10 s, 축만 바꿨다

Yaw

ψ 가 118° 돌고 궤적이 흰다.
고도는 300 m 그대로.

Pitch

θ 가 들려 7.55 km 상승.
지상 궤적은 거의 제자리.

Roll

ϕ 만 변하고 궤적은 불변.
회전축이 곧 속도 방향이라 그렇다.

Roll 이 궤적을 못 바꾸는 것은 구현 오류가 아니라 물리적으로 옳은 결과다.

01. 표적을 정의한다

코드로 보는 구조체

입력 — TargetSim.h

```
// 기동 1 건 . gravityValue 의 부호가 회전 방향이다 .
typedef struct
{
    EN_TurnType    enTurnType;    // 0~3
    FLOAT64        gravityValue;  // [G]
    FLOAT64        startTime;     // [s]
    FLOAT64        endTime;       // [s]
} ST_TargetManeuver;

// 위치는 위 / 경 / 고도 , 속도는 동체 x 축 방향 크기 .
typedef struct
{
    STRUCT_Coord_Lla    st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64             headingSpeed;  // V_heading
    INT32               nManeuverNum;
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

출력 — TargetSim.h

```
// 한 스텝 계산은 ECEF 에서 , 출력은 LLA 로 .
// 둘을 함께 들고 있어 스텝마다 한 번만 변환한다 .
typedef struct
{
    FLOAT64        simTime;    // [s]
    STRUCT_Coord_Rect st_PosEcef;  // [m]
    STRUCT_Coord_Rect st_VelEcef;  // [m/s]
    STRUCT_Coord_Lla st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 · 표적 상태 — 이게 한 장의 표본이다 .
typedef struct
{
    FLOAT64        simTime;
    INT32          nStepIndex;
    ST_TargetState st_Platform;
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

구간은 [시작 , 종료)

끝과 다음 시작이 같아도 겹침이 아니다.
실제로 겹치면 계산 전에 거부한다.

계산용과 출력용을 함께

계산은 ECEF, 사람이 읽는 값은 LLA.
스텝마다 한 번만 바꿔 두 곳에서 쓴다.

플랫폼도 같은 구조체

기동이 없는 표적일 뿐이라 , 각속도만 0 으로
넣어 같은 경로로 전진시킨다.

02. 시간에 따라 움직인다

한 스텝에 하는 일

이 다섯 단계가 `f_Tgt_StepSim` 한 번입니다. 601 번 반복하면 60 초 궤적이 됩니다.

0.1 초를 전진시키는 다섯 단계

이 다섯 줄이 `f_Tgt_StepSim` 한 번이다. 601 번 반복하면 60 초 궤적이 된다.



③④ 는 실제로 두 번 돈다 — 중점법

반 스텝만 살짝 가서 그 지점의 속도를 다시 갱 뒤,
그 속도로 한 스텝을 옮긴다. 굵은 길일수록 이 쪽이 맞다.

표적 수만큼 이 다섯 단계를 돌린다

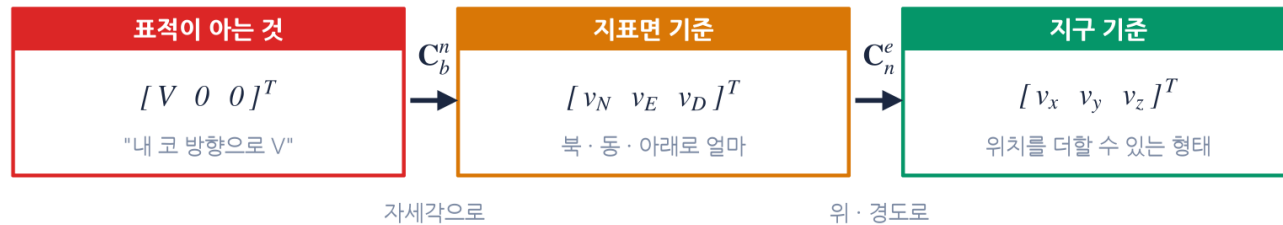
표적끼리 서로 영향을 주지 않으므로
for 문 하나로 10 대까지 그대로 늘어난다.

02. 시간에 따라 움직인다

공식 ① — 속도 벡터를 만든다

과제가 따로 묻은 "동체 속도 V_{heading} 을 ECEF 의 V_x, V_y, V_z 로 바꾸는 방법" 에 대한 답입니다.

속도 벡터 만들기 — 받은 것은 숫자 하나, 필요한 것은 방향까지



$$\mathbf{v}^e = \mathbf{C}_n^e(\phi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

왜 이 식이 필요한가

- 입력은 속력 하나뿐이다 — 방향은 자세각이 들고 있다.
- 위치를 더하려면 위치와 같은 좌표계의 속도여야 한다.

두 회전행렬은 Part 2 에서 만든 함수를 그대로 가져다 썼다.
새로 만든 것은 "두 번 이어 붙인다" 는 이 한 줄뿐이다.

이 식이 맞게 돌아갔다는 근거

$$\|\mathbf{v}^e\| = V$$

회전은 방향만 바꾼다 — 601 표본 전부
입력 속력과 같게 나왔다.

함수 한 줄에서 주의한 곳

```
f_Trans_Ned_To_Ecef(  
    vN, vE, vD,  
    0.0, 0.0, 0.0, lat, lon)
```

이 함수는 기준점까지 더해 준다. 속도는 방향만
옮기면 되므로 기준점을 0 으로 넣는다. 위치를
옮길 때와 호출 방법이 다른 유일한 지점이다.

02. 시간에 따라 움직인다

코드 ① — 속도 벡터를 만든다

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번 , 위경도로 한 번 돌려 ECEF 속도로 만든다 .
// 회전만 거치므로 속력은 그대로 보존된다 .
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
    const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다 .
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
        st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다 . 방향만 옮길 때는 기준점을 원점으로 준다 .
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
        0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

기준점을 0 으로 주는 이유

이 함수는 회전에 더해 기준점만큼
평행이동까지 한다. 속도는 "위치" 가 아니라
"방향" 이라 그 이동분을 빼야 한다.

인자 순서를 주석에 남긴 이유

제공받은 함수가 Roll, Yaw, Pitch 순서로
받는다. 자세각 구조체의 필드 순서와 달라
한 번 틀렸던 자리다.

제대로 돌아갔는지 확인

회전은 방향만 바꾸고 크기는 건드리지
않는다. 601 표본 전부에서 속력이 입력값과
같게 나오는 것으로 확인했다.

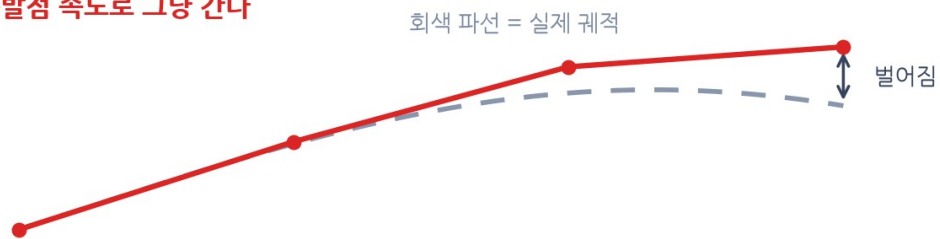
02. 시간에 따라 움직인다

공식 ② — 한 스텝 전진시킨다

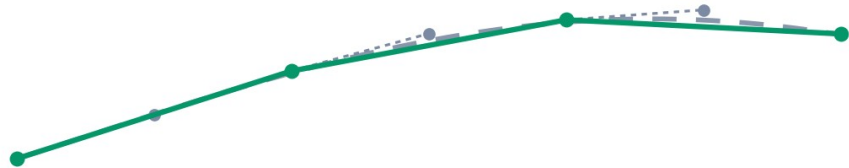
어느 지점의 속도를 쓰느냐로 결과가 갈립니다.

한 스텝 전진 — 어느 지점의 속도를 쓸 것인가

출발점 속도로 그냥 간다



반 스텝 가서 속도를 다시 재고 간다



회색 점이 반 스텝 지점이다. 그 자리에서 잦 속도가 구간 전체의 평균에 훨씬 가깝기 때문에, 같은 간격으로도 덜 벌어진다.

쓰는 식

$$\mathbf{v}_{mid} = \mathbf{v}\left(\mathbf{p}_k + \frac{\Delta t}{2}\mathbf{v}_k, \quad \Theta_k + \frac{\Delta t}{2}\boldsymbol{\omega}\right)$$

$$\mathbf{p}_{k+1}^e = \mathbf{p}_k^e + \mathbf{v}_{mid} \Delta t$$

반 스텝 지점의 위치와 자세로 속도를 한 번 더 만들어 쓴다.

왜 굳이 두 번 계산하는가

표적은 늘 돌고 있고, 지구도 둥글다.

한 스텝 안에서도 방향이 계속 바뀐다.

같은 0.1 초 간격에서 60 초 뒤 위치 오차

출발점 속도로

16.6 m



중점 속도로

1.36 cm

약 1,200 배

02. 시간에 따라 움직인다

코드 ② — 한 스텝 전진시킨다

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발췌)

```
halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세 — 각속도가 구간 내 상수라 바로 더한다
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...
```

자세만 반 스텝 돌리면 안 되는 이유

"북쪽"이라는 기준은 표적이 서 있는 자리에서 정해진다. 자세만 돌리고 위치를 두면 방향의 기준이 뒤에 남아, 곧게 날려도 고도가 조금씩 뜬다. 위치도 함께 반 스텝 옮긴 이유다.

계산이 깨져도 상태는 지킨다

사본에서 전부 계산하고 값이 정상인지 확인한 뒤, 모두 성공했을 때만 한 번에 반영한다.

알고도 남겨 둔 한계 — 짐벌락

Pitch 가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐 Yaw 기동이 궤적을 바꾸지 못한다.

03. 결과를 확인한다

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60 s · 간격 0.1 s

스텝 600 회 , 표본 601 개 .

시각은 이전 값에 더해 가지 않고
스텝 번호 × 간격으로 매겨
오차가 쌓이지 않게 했다.

답을 미리 알 수 있는 조건이다

기동이 하나도 없다 . 그래서 손으로
답을 낼 수 있다 .

대함은 정서진 → 위도 그대로 ,
대공은 정남진 → 경도 그대로 .

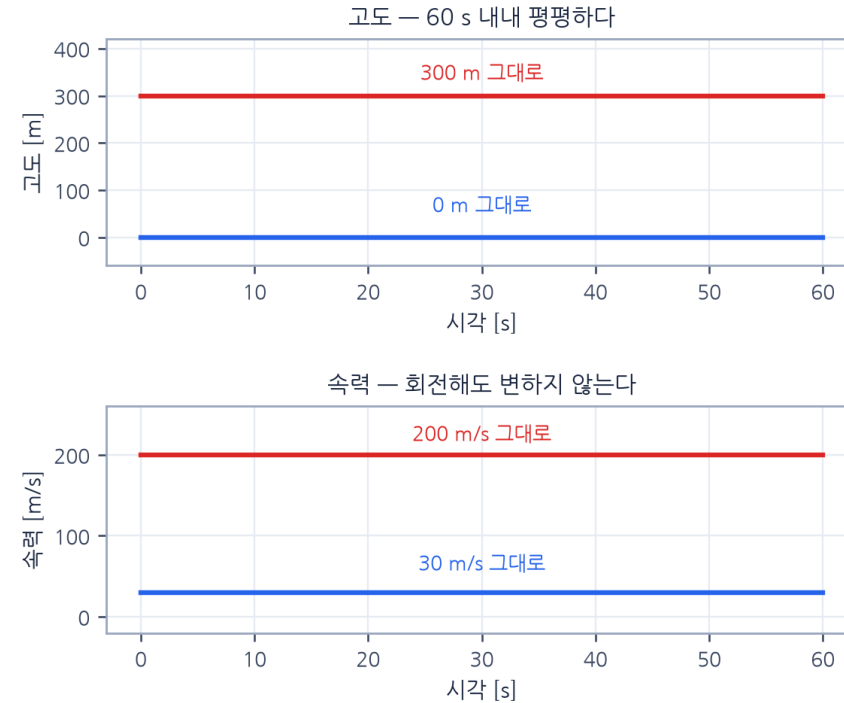
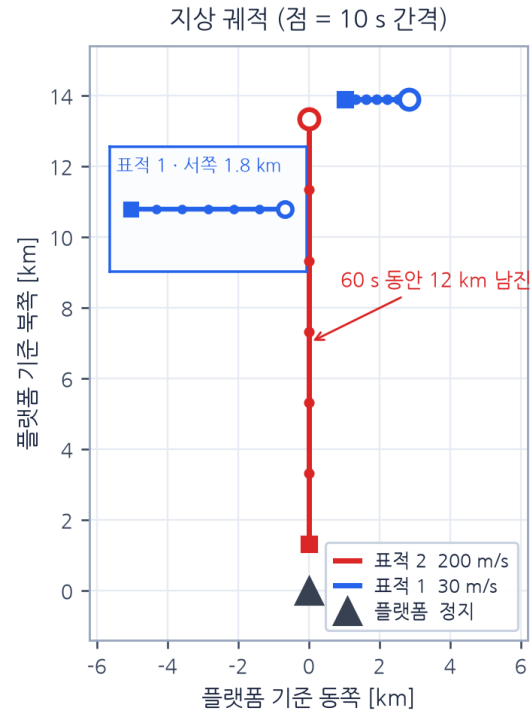
플랫폼 속력이 0 인 점

각속도 식은 속력으로 나눈다 .

그 나눗셈은 기동이 걸린 구간
안에서만 하는데 , 플랫폼은 기동이
없어 그 자리를 지나가지 않는다 .

03. 결과를 확인한다

결과 ① — 명세 조건 60 초



이동 거리

대함 1800 m · 대공 12000 m —
속력 × 시간과 같다.

방향

대함은 위도가, 대공은 경도가
60 초 내내 그대로였다.

고도 · 속력

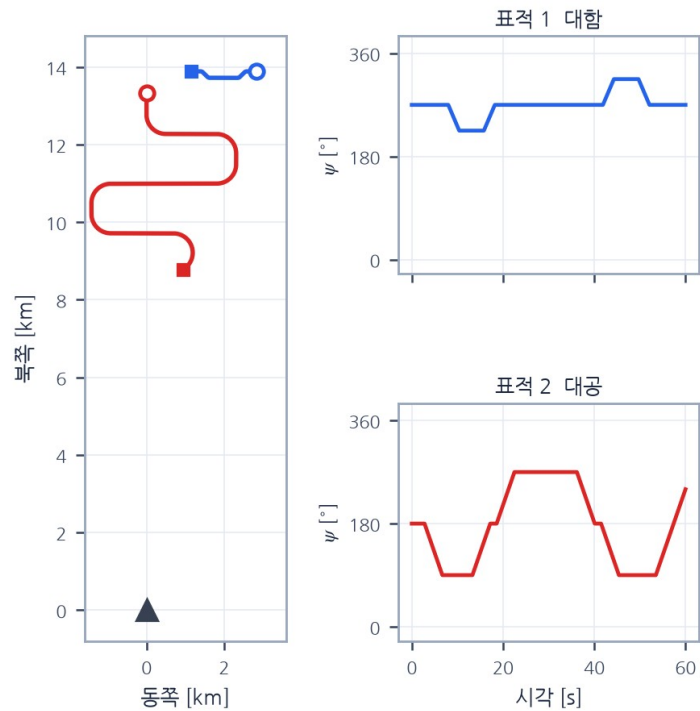
널은 값이 끝까지 유지됐다 —
회전이 크기를 바꾸지 않는다는 뜻.

03. 결과를 확인한다

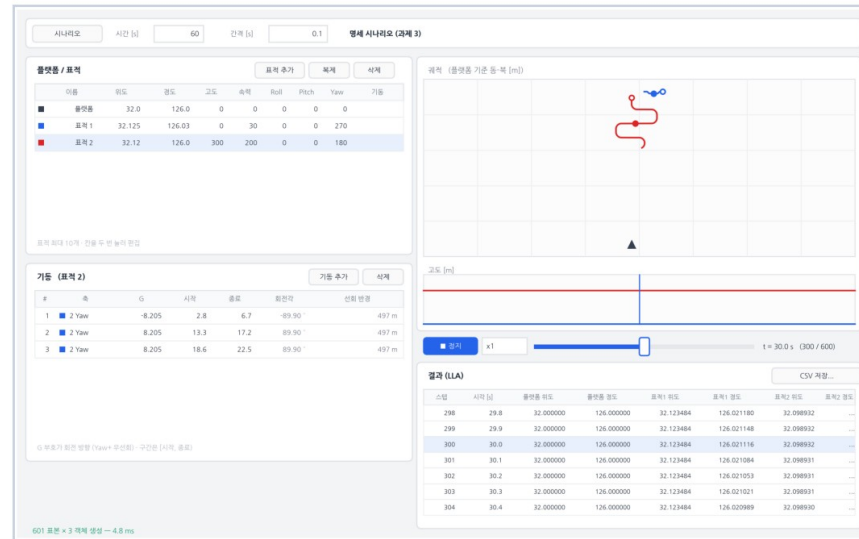
결과 ② — 기동 시연과 운용 화면

초기값 · 시간 · 간격은 명세 그대로 두고, 기동표 10 줄만 추가했습니다.

기동 10줄을 얹은 60 s 궤적



TargetSim — 값을 바꿔 가며 확인하는 화면



고치면 곧바로 다시 돈다

표의 칸을 고치면 0.25 s 뒤 자동으로 다시 계산한다. 601 표본 × 3 객체가 4.8 ms 라 기다릴 일이 없다.

그림에서 끌어서 배치

시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 반영된다. 회전각과 선회 반경은 G 로부터 계산.

결과는 표 · CSV · 그림

시각별 LLA 를 표로 보고 슬라이더로 되감고 CSV 로 내보낸다. 오늘 숫자가 전부 여기서 나온 CSV 다.

03. 결과를 확인한다

정리

요구사항과 구현을 한 장에 놓고 보겠습니다.

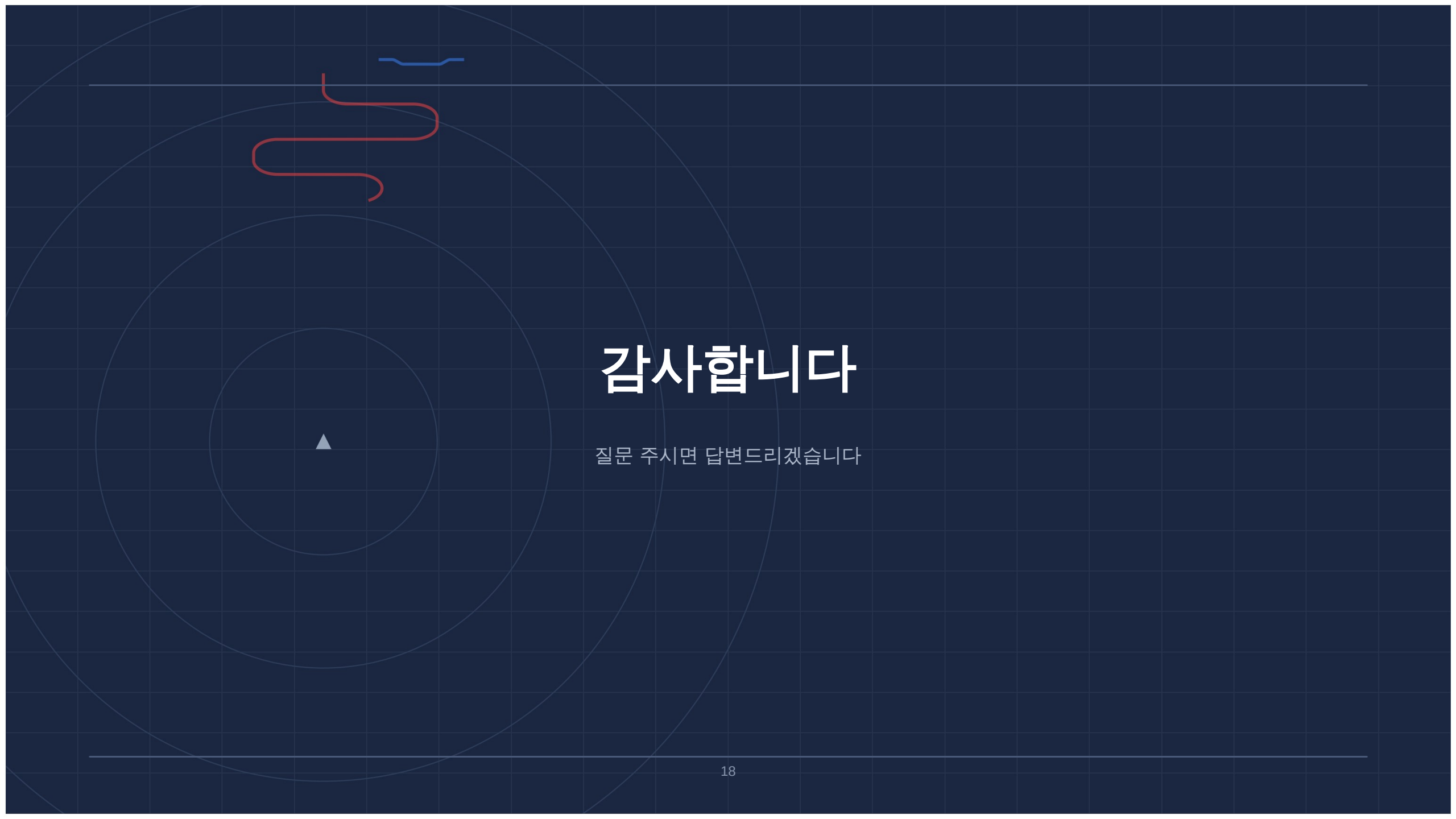
과제 요구사항	구현	확인
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_{heading}	ST_TargetInit — 방향은 자세각이 담당	통과
기동 = $G \cdot \text{축} \cdot \text{시작} \cdot \text{종료}$	ST_TargetManeuver 4 필드, 구간 겹침 검사	통과
ECEF 에서 상태 갱신	위치 · 속도는 ECEF, 출력용 LLA 병행 보관	통과
$V_{\text{heading}} \rightarrow V_x, V_y, V_z$ (추가 고민)	동체 $\rightarrow$ NED $\rightarrow$ ECEF, 회전 두 번	속력 보존
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본	무수정
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 · 고도 · 방위 그림	601 표본

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 — 사원수로 바꾸면 해소된다
- 힘이 아니라 속도를 직접 주는 모델 — 탄도 표적에는 부적합
- 속력이 일정해 가감속 기동은 아직 표현하지 못한다

이 위에 이어 붙일 것

- 궤적에 측정 잡음을 없으면 그대로 추적 필터 입력이 된다
- 안테나 기준 거리 · 방위 · 고각으로 바꾸면 탐지 모의가 된다
- 참값을 알고 있으니 추적 오차를 바로 숫자로 낼 수 있다



감사합니다

질문 주시면 답변드리겠습니다